Memglide: Auditable, Bounded Repository Context for Coding Agents

SREE CHARAN REDDY KAILASAM

Repository context can reduce a coding agent's search work, but selection, verification and added prompt text also have costs. Memglide prepares bounded current-source context using lexical and static syntax evidence, abstains when selection is ambiguous, and optionally supplies explicitly verified public commands and baseline counts. We evaluate its installed launcher against pass-through Codex and source-only preloading on twelve custom feature tasks in newly pinned public repositories. A prospectively frozen balanced design assigns three arms twice per task, producing 72 native workflows. All 72 pass finite feature, original public-check and integrity requirements, with complete usage telemetry. The source-and-command workflow has 13.69% lower observed uniform token-tariff-equivalent cost than pass-through and 10.04% lower cost than source alone. The exact conditional tests yield Holm-adjusted p=0.148 for both contrasts; neither establishes a reduction under the prespecified rule. Mean workflow times are 40.56, 39.00 and 40.03 seconds for pass-through, source-only and source-and-command modes. Source activates on seven tasks and abstains on five. The implementation and evidence support a functioning, auditable context policy, while the small task-informed panel, finite checks and uncontrolled caches limit claims about general savings or speed.

CCS Concepts: • **Software and its engineering** → **Software testing and debugging**; • **Information systems** → *Information retrieval*.

Additional Key Words and Phrases: coding agents, repository context, source retrieval, reproducibility, software engineering

1 Introduction

Coding agents must find the source and test commands needed for a change. Providing that information in advance may reduce their work, but preparation also costs time and adds to the prompt. The relevant question is whether the complete workflow improves after selection, verification, prompt length, failed attempts and preservation of existing behavior are counted. A compact source bundle or shorter serialized response cannot settle that question.

Memglide is a local repository-context system with a native coding-agent launcher. The launcher uses lexical and static syntax evidence to select current source, delivers complete selected files within fixed limits, and preserves the original task when the evidence is insufficient. An optional workflow supplies public test commands and observed baseline counts from an explicitly verified receipt. Preparation uses no auxiliary language model or embedding service. The external coding model interprets the task, edits the code and runs checks.

Earlier experiments revealed gaps between a working context system and an efficiency claim. Interactive MCP retrieval could increase tokens and elapsed time. An automatic selector could abstain on most tasks, producing apparent agent differences without evidence of a preload effect. A development-only compiler configuration could change a score in a separate compatibility diagnostic. A provider failure could leave token usage unknown. These findings motivated a comparison of the installed system with explicit records of activation, source protection, timing and missing data.

Development included a 30-round search over prompt and verification approaches on three tasks, followed by a two-arm comparison on eight new tasks. We treat that work as exploratory. The new study compares pass-through Codex, source preloading, and source plus verified public-command context. Two repeats on twelve newly selected public repositories yield 72 scheduled workflows. Private grading is deferred until native collection closes. The private reference implementations, oracles and revealing plaintext logs are unavailable to the coding agent during collection.

The tasks are author-designed feature additions to small public libraries. SWE-bench, by comparison, evaluates patches for real GitHub issues [2]. Public task readiness informed a small production-file classification fix before

Author's Contact Information: Sree Charan Reddy Kailasam, srcharan014@gmail.com.

this comparison. Coding outcomes were unobserved at the freeze, but the retrieval algorithm had already received task-informed engineering. The study is therefore a prospective installed-system ablation. Its small, purposive panel limits generalization to other coding workloads.

We ask three questions. First, when does the installed policy deliver source and command context, and when does it abstain? Second, how do the modes compare under finite feature, original public-check and source-integrity requirements? Third, how do native token-tariff equivalents and workflow time change when fresh indexing and public baseline verification are included?

The contributions are an implementation of bounded context preparation with verifiable receipts; an evaluation method that separates activation, functional outcomes, timing and unavailable telemetry; and a development-to-evaluation record that retains unsuccessful approaches. The study does not introduce new model weights, measure memory-reuse benefits or establish a benchmark-leading coding agent.

2 Related work

Repository context has been studied as both a retrieval problem and part of code generation. RepoCoder combines a similarity retriever with a pretrained code model in an iterative retrieval–generation pipeline [7]. RepoBench evaluates retrieval, completion and a combined pipeline, including cross-file dependencies [4]. These settings separate the evidence a retriever returns from the result of using it. Memglide evaluates functional patches with preserved public regressions on small feature tasks, rather than code-completion similarity.

Context length alone gives an incomplete account of useful input. Liu et al. varied the position of relevant information in multi-document question answering and key–value retrieval. Performance often declined when relevant information appeared in the middle of long contexts [3]. LLMLingua compresses prompts through a coarse-to-fine method with token-level compression and model alignment [1]. Memglide uses lexical evidence and static syntax analysis to construct bounded bundles of selected complete files. Preparation makes no auxiliary model call and can abstain when selection or freshness checks fail. The present study includes no performance comparison with these approaches.

Agent design provides a second point of reference. SWE-agent studies an agent–computer interface for automated software engineering [6]. Agentless separates localization, repair and patch validation [5]. Memglide runs before the installed coding agent, supplying source and optional verified-command context while leaving investigation and editing to the agent. SWE-bench evaluates issue-to-patch behavior on real GitHub issues [2]. The present panel uses author-designed extensions to public repositories. Its results describe an installed-system ablation within that panel; they supply neither a SWE-bench score nor a direct performance comparison with these systems.

3 Architecture and evidence boundaries

Memglide prepares repository context and records the source and policy used to prepare it. Its central design choice is conservative selection from current local files. Lexical evidence, static syntax and bounded dependency expansion produce a small source bundle without an embedding model or auxiliary language-model request. The coding agent interprets the task, edits the repository and checks its work. Memglide changes context preparation rather than model weights.

The implementation has two entry paths. Interactive clients reach a common service through a command-line interface, loopback HTTP API or MCP STDIO server. The service combines repository indexing, context retrieval, a SQLite database and explicit session memory; a dashboard uses the HTTP API. The native launcher, `memglide-codex`,

calls the source-preparation libraries directly before executing Codex. The experiment measures this prelaunch path, shown in Figure 1. Interactive retrieval, the dashboard and session-memory reuse require separate evaluation.

Repository registration resolves the Git root and records project state. The SQLite FTS5 index stores searchable source, file hashes and metadata, symbol locations and signatures, and conservative dependency edges. Python structure comes from the standard AST; JavaScript and TypeScript use an isolated Tree-sitter worker. Incremental refresh checks current files before using indexed evidence. Dynamic imports, generated code and runtime dispatch can leave static dependencies unresolved, limiting what the selected context represents.

The interactive context engine combines identifier and path matches, normalized terms, FTS5 evidence and graph expansion. Session memory stores caller-submitted observations, decisions, errors and fixes with provenance, timestamps and source versions. Sessions have inactivity and absolute expiry limits. Returned context passports identify source hashes, memory versions and omissions. These features provide versioned context within an active session; the experiment does not measure cross-session reuse. The native prelaunch selector follows a separate deterministic algorithm and does not automatically inject session observations.

Prelaunch selection first checks whether preparation is enabled, context has already been supplied, and an approved project and eligible task are available. Ineligible requests retain the exact original task. The v4 policy then securely refreshes the index and screens production files. Known tests, examples and demonstration directories are excluded from production competition. Potentially relevant unsupported files, partial indexing or an overfull candidate frontier cause abstention. Coverage is therefore an observable policy outcome: reading source successfully does not establish that the task received useful context.

Remaining supported candidates are parsed into runtime or declaration syntax units. Normalized query terms receive candidate-set frequency weights $w(t) = \log(1 + (N + 0.5)/(\mathrm{df}(t) + 0.5))$. Here N is the number of files passed to syntax ranking, and $\mathrm{df}(t)$ counts those files with term t in the clean body or header of any eligible-role syntax unit. A file counts once even if several units match; this frequency is computed on the bounded candidate set, rather than the entire repository. A unit is sufficient if it has an explicit current anchor, or at least two distinct clean body/header matches including one body match. Explicit and sufficient evidence receive large priority terms in ranking. The remaining score combines body, header, path and documentation weights with coefficients 3, 2, 0.25 and 0.05. This score orders evidence; it is not a calibrated confidence. An independently competitive file causes abstention unless the leader has the required body-evidence dominance.

After accepting an anchor, Memglide reads the complete file and at most three resolved direct production helpers. It checks their current hashes and selection evidence before rendering the task. “Full source” refers to these selected files, not repository-wide or transitive dependency coverage. Final handoff revalidates inspected competitors, selected source, approval and ignore policy. A changed proof prevents reuse of the prepared context. Source, comments and strings remain project data for the coding agent to interpret.

Each phase has its own bounds. Default indexing permits 20,000 files and 100 MiB of eligible source, with a 512,000-byte per-file read limit. The v4 task limit is 4,000 UTF-8 bytes and 64 normalized non-generic terms, each at most 128 characters. Coarse screening permits 100 files and 2 MiB; syntax analysis permits 24 files and a separate 2 MiB. The syntax worker has a five-second timeout, 200,000-node limit and 1,000,000-byte output limit. Added context defaults to 12,000 UTF-8 bytes, with an additional `ceil(bytes/3)` planning limit of 4,000. This proxy is not a provider token measurement. Repeated freshness checks mean that phase byte limits are not whole-workflow I/O or memory bounds.

The selected workflow adds public-command assistance through explicit verification. The caller declares atomic argument lists and allowed environment updates. A verifier records a fresh bounded public-text tree, fingerprints

declared executable files, runs the checks, and records exit status, positive behavioral-test counts and log hashes. It checks the tree, policy and tools again before writing a canonical receipt outside the project. The caller retains the trusted receipt hash. Because public tests may execute repository code, verification is a separate authorized action; context preparation does not itself run those commands.

The launcher loads the pinned receipt, checks its current bindings, and places the declared commands and observed baseline results beside the selected source. Allowed environment updates appear explicitly in the card. These results concern the starting repository, before the feature is implemented. Stale, unavailable or oversized optional facts leave ordinary source preparation available. An invalid explicitly pinned receipt on an active workflow path is an error. Before native execution, the launcher repeats source selection and checks the exact rendered transformation. Abstention preserves the task bytes.

The trust boundary separates local preparation from an external coding model. Selected source is sent to Codex; local-first describes where the index and state reside. Secure readers enforce containment, reject symlink traversal and read bounded regular UTF-8 files. Known sensitive filenames and content patterns are filtered heuristically. Hashes establish equality at checked reads, not semantic safety, permanent freshness or universal secret detection. A command receipt covers eligible public text and declared executable bytes, not every transitive dependency, binary resource or undeclared environment setting. Command-runner time and output limits also differ from operating-system isolation.

These boundaries motivate separate evaluation of activation, functional success, preparation time, native execution time and token-tariff equivalents. The source-and-command workflow incurs verification and freshness costs. Finite public and hidden checks do not establish universal correctness, semantic quality of new tests or general coding superiority. Natural abstentions and unknown telemetry remain in the record. In particular, the base58 oracle's output-bound checks cannot prove the requested allocation order. Launcher results do not establish benefits for unmeasured memory, API or dashboard features.

4 Evaluation design

4.1 Tasks and prospective readiness

The panel contains one author-designed feature addition in each of twelve newly pinned public repositories: six Python projects and six JavaScript/TypeScript projects. Two of the latter tasks require only JavaScript changes; four require runtime and declaration changes. Four JavaScript/TypeScript roots share a maintainer. The projects are small and purposively selected for executable upstream checks. They are not a sample of historical issues.

Repository provenance, commit hashes, retained public inputs, prompts and commands are committed before native outcomes. This local hash freeze is not a public preregistration. Screening records retain acquisition and compatibility rejections, including an obsolete Python testing dependency and an incompatible upstream Node execution setup. Repository novelty is relative to the earlier Memglide evaluation roots; the model's training data are unknown.

Tasks and checks were developed with AI assistance. Separate Codex agents proposed prompts and prepared feature oracles, reference implementations and plausibly incorrect controls. Another agent reviewed task–oracle agreement and control records. These are separate authoring roles, not independent human replication. Baseline, reference and incorrect variants across twelve tasks provide thirty-six expected control outcomes, with positive original public-test counts: the baseline fails the feature, the reference passes, and the incorrect implementation can fail without changing upstream tests. These controls do not establish exhaustive coverage. Original tests, configurations and provenance are retained; public declaration compilation is enforced where applicable. Regression tests are requested, but their presence

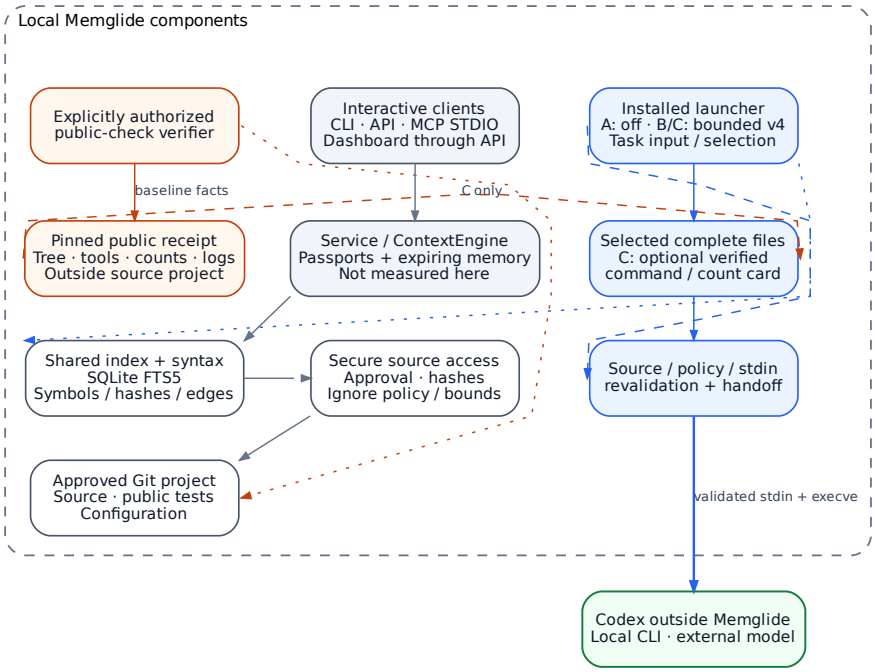


Fig. 1. Memglide components and the evaluated installed path. Blue denotes native prelaunch; orange denotes separately authorized public verification and its receipt. Gray interactive services and session memory are implemented but unmeasured. Native discovery accesses the shared index directly. Pass-through and abstention preserve the task. The dashed rectangle is a component boundary, not an operating-system sandbox; Codex uses a local CLI and an external model.

and execution are separate diagnostics rather than part of the primary functional gate. For base58, black-box checks establish output bounds and error behavior, not the requested order of allocation and validation.

Public readiness exposed source-role classification errors involving root-level test files and demo directories. Minimal shared B/C fixes and independent production-competitor controls were completed before any new native outcome. This task-informed engineering means the panel is not a blinded test of algorithm generalization. Queries, ranking thresholds and context caps were not adjusted to force preloading. Seven tasks preload at readiness and five abstain because of candidate ambiguity. All twelve remain in the intention-to-treat panel. The frozen activation strata are descriptive, with no subgroup hypothesis tests or replacement of the primary panel.

4.2 Conditions, allocation and execution

Arm A runs the installed launcher in explicit off/pass-through mode. Arm B enables v4 discovery and compact source context. Arm C uses the same source policy plus the selected verified public-command card. C versus A measures the

overall installed intervention; C versus B compares the added card and fresh-verification workflow. Abstention remains a policy outcome, and its indexing and verification overhead belongs to the assigned workflow.

Each task receives all three arms twice: seventy-two scheduled workflows and twenty-four cells per arm. Before outcomes, seed 20261027 shuffles the task sequence and a multiset containing each of the six arm orders twice. Repeat two reverses the entire first-repeat task/arm sequence. Assignments run sequentially, each with a fresh project copy, native state and thread. The model alias is gpt-6.1-sol, with low effort and the default service tier, using Codex CLI 0.159.2. Each launcher receives the same feature-request stdin and developer instructions. Native prompts differ by assigned context; response schema, tools and network-disabled workspace-write configuration are shared. Project-document injection is disabled identically. Thirty-six installed zero-model handoffs establish expected treatment before sealing. Actual prompt bytes and arguments are captured during native execution and checked against that commitment.

Session counts refer to the coding-agent `codex exec` evaluation path. Authoring controls, handoff checks and public/private graders can use local `codex sandbox` processes without making model requests; these processes are not additional coding-model sessions. Neither CLI process counts nor observed agent commands reveal internal provider inference counts.

Each assignment is allocated durably once. There are no retries, replacements or outcome-driven repeats, and the controller admits only the next unused assignment. Missing whole-attempt usage, lifecycle failure or integrity failure causes a durable stop; unrun cells remain unavailable. The maximum is seventy-two scheduled native dispatches. A soft USD 18 allowance uses the uniform accounting convention, with a USD 0.25 reserve before another call is admitted. It bounds neither account charges nor the cost of an admitted call.

4.3 Timing, isolation and grading

Public-source copying and Git-baseline construction occur outside every workflow timer. Timing starts before arm-specific registration, indexing or verification and ends after native process-group cleanup. C includes public baseline verification and removal of its newly generated logs, build outputs and caches. Original source and Git bytes, and the provider manifest, are checked afterward. This cleanup is an experimental control, not automatic shipping behavior. Component measurements separate indexing, verification, cleanup, launcher preparation and native execution. An absolute ninety-second setup allowance covers setup through launcher preparation. Native execution receives a separate three-hundred-second allowance from the recorded local bridge handoff. Native wall time includes final local exec setup and cleanup; it does not isolate remote inference latency. Grading and evidence packaging are excluded.

All arms use a frozen environment profile, including TERM=xterm and disabled supported runtime caches. Projects and run caches are fresh. Operating-system page caches and provider prefix caches are uncontrolled; observed cached input is reported. No unrelated heavy validation runs during measured execution.

Before launch, private grader/reference/control bundles and revealing logs are encrypted and authenticated with AES-GCM. A random 256-bit key remains only in the external orchestrator store. Ciphertext and hash commitments may remain locally, but every private plaintext root is absent before dispatch. This sequesters the new oracle plaintext; it is not an operating-system isolation claim. Exact committed bytes are restored after native closure or a durable stop.

Private and public grading then runs with candidate source read-only, network disabled and only grader artifacts writable. Inherited global temporary write roots are disabled. Source and protection hashes are checked before and after grading. Strict success requires valid native completion and usage, treatment fidelity, intact protected inputs, valid grading infrastructure, and passing committed feature and original public checks.

4.4 Accounting and statistical analysis

The two primary contrasts are C minus A and C minus B in uniform API-equivalent dollars. Repeated costs are averaged within each task before computing its paired arm difference. The fixed convention is [(input minus cached input) × 5 + cached input × 0.20 + output × 15] / 1,000,000. This historical tariff convention is neither an invoice nor a current-price claim. Ordinary coding failures retain observed costs; unknown usage is never replaced with zero. A complete-panel estimate requires all twelve task clusters and both repeats for the contrasted arms. Otherwise the primary total, effect and test are unavailable, and known subtotals and missing identifiers are reported. Actual billing and internal provider request counts are unavailable.

Complete contrasts receive pointwise 95% paired task-bootstrap intervals from 10,000 resamples, using seed 20261029. Resampling retains both repeats and paired arms. For exact tests, conditioning on the third arm's first-repeat position creates three strata of four tasks, each containing two relative orders of the compared arms. Enumerating choose(4,2)^3 = 216 legal allocations, with joint repeat-label swaps, respects the balanced design. Two-sided tests count ties and require a pairwise sharp null and no cross-arm/provider-cache carryover. Holm adjustment covers the two predeclared cost tests; missingness does not reduce the family. Native-price/cache-write sensitivities, tokens, workflow components, readiness strata and joint-success timing are secondary descriptive analyses. Command-text read/test counts are heuristic. Cheaper failures cannot establish efficiency, and matching observed success counts does not establish population quality equivalence.

The study is precision limited. Planning from seven earlier complete two-repeat task clusters gives an unstable cost-difference standard deviation of USD 0.03441. An optimistic normal approximation suggests a detectable difference near USD 0.03063, about 25% of the earlier A mean, for twelve clusters at 80% power and first-step two-sided alpha 0.025. This is not achieved power or a guarantee for the exact test. Earlier adaptive prototype results remain exploratory and are not pooled with the installed-system panel.

5 Development history

The final configuration emerged through unsuccessful and partly successful evaluations. The supplement inventories 23 historical stages and their reports. Some rows reanalyze earlier attempts and must not be counted as new model sessions. The stages explain the implementation and measurement choices; their different conditions and task sets do not form a common representative benchmark.

Early native MCP experiments showed that retrieval could be costly. In one matched cohort, the context-assisted condition used 2.751 times the input tokens and 1.465 times the Codex-plus-index time. An optimized cohort still increased input by 32.82% and Codex-plus-index time by 17.88%, although both conditions passed their finite checks. Known-path source preloading later reduced some aggregate measures, without establishing automatic discovery or a joint cost and speed improvement. A partial memory study also failed to establish a warm-memory benefit.

Source delivery needed its own verification. Earlier automatic policies often abstained or selected an incorrect target in local tests. In a fresh v4 native comparison, the automatic condition added source on none of its 24 assignments, leaving task prompts identical to the pass-through baseline. Agent differences in that cohort cannot establish a source-preloading effect. A subsequent active pilot delivered source, but one failure changed a lower observed cost per attempt into a higher cost per strict success. A feedback-informed repair was retained as development evidence, without replacing the original attempt.

Table 1. All assigned attempts. A: pass-through; B: source; C: source and command workflow. Costs are means under the fixed uniform tariff.

Arm	Strict pass	Uniform USD	Workflow (s)	Native (s)
A	24/24	0.132553	40.564	40.450
B	24/24	0.127168	39.004	38.503
C	24/24	0.114403	40.025	37.909

The v5 search tested 30 candidate workflow changes on three development tasks: three reference sessions and 90 candidate sessions. It varied command cards, baseline counts, public-test inclusion, regression instructions, compatibility review, output handling, context layout, and other prompt or verification choices. Model weights were unchanged. The first candidate added verified public commands; the second added observed baseline counts. Both were promoted, and the remaining 28 were rejected under the frozen development rules. The selected configuration retained ordinary testing and review instructions and omitted public test bodies from added context. The supplement records each round's hypothesis, outcome, resource measurement and decision.

A separate v5 comparison scheduled two conditions and two repeats on eight new tasks, for 32 attempts. Both conditions passed 15 of 16 strict assessments. One pass-through patch passed behavioral checks but added a test that broke the repository's original TypeScript compilation. One assisted attempt ended with a provider-capacity error and unknown usage. Neither was replaced by a successful retry.

Among the seven task clusters with complete usage, the assisted condition's uniform token-tariff equivalent was 18.726% lower. The mean paired difference was −0.02269 dollars per attempt, with a 95% task-bootstrap interval from −0.04695 to +0.00105 dollars. This complete-case result did not establish a general cost reduction. A secondary native-price convention produced a different interval and retained its secondary status. Account billing was not observed.

The v5 timer omitted public baseline verification required by the installed workflow, and its prompt-generation framework was broader than the selected shipping modules. The new study therefore measures the installed launcher and includes fresh verification and cleanup in workflow time. It also sequesters all new private oracle plaintext during collection. Earlier results remain bound to their original source snapshots and scoring rules; the new classification fix does not rescore them.

6 Results

All 72 scheduled assignments completed with valid native telemetry and complete whole-attempt usage. Each arm passed 24 of 24 strict assessments. There were no unavailable outcomes, protected-file violations or grading-infrastructure failures. Before candidate grading, all 36 author controls were rechecked through the final frozen sandbox: references passed the feature checks, baselines and deliberately incorrect variants failed them, and original public checks passed throughout. These outcomes show that all submitted patches passed the committed rubric on this panel; the saturated scores cannot establish population quality equivalence or distinguish the arms' general coding ability.

Table 1 reports all assigned attempts. Workflow time includes preparation and cleanup; native time begins at the recorded local bridge handoff. Costs use the fixed uniform tariff and are not account charges.

Table 2. Complete-panel primary contrasts: mean dollar difference per attempt, relative change, pointwise bootstrap interval and prespecified exact tests.

Contrast	Delta, USD	Change	95% CI, USD	Exact p	Holm p
C–A	−0.018150	−13.69%	[−0.035594, −0.000315]	0.0741	0.1481
C–B	−0.012765	−10.04%	[−0.027778, +0.001526]	0.1389	0.1481

6.1 Primary cost contrasts

The complete-panel uniform equivalents total USD 3.181276 for A, 3.052038 for B and 2.745678 for C. Thus the observed C total is USD 0.435598 lower than A and USD 0.306360 lower than B over 24 attempts. No cost is imputed. Across twelve paired task means, the C-minus-A estimate is -0.018150 dollars per attempt (-13.69%); C-minus-B is -0.012765 dollars (-10.04%).

Table 2 retains the two prespecified tests. The exact conditional tests count 16 and 30 extreme allocations, respectively, among 216 legal allocations. Neither rejects at familywise alpha 0.05 after Holm adjustment. The C-minus-A bootstrap interval is below zero, but it is a pointwise task-resampling interval with different assumptions from the design-based test. It does not supersede the fixed testing rule or establish general savings. Figure 2 shows the task variation behind the aggregate estimates.

6.2 Time and preparation overhead

For workflow wall time, C minus A is -0.539 seconds per attempt (-1.33%), with a pointwise 95% task-bootstrap interval of [-4.648, +2.778] seconds. C minus B is +1.021 seconds (+2.62%), with an interval of [-0.903, +2.940] seconds. These descriptive intervals do not establish a workflow speed improvement. Because all attempts pass, the prespecified jointly successful timing subset contains the same twelve tasks and produces the same values.

C's mean native time is 2.540 seconds below A and 0.594 seconds below B. Fresh public verification takes a mean 1.363 seconds per C assignment, while indexing takes 0.101 seconds and launcher preparation 0.425 seconds. Verification cleanup and other local overhead are also counted. Figure 3 separates these components. The source-and-command workflow's shorter observed native time is therefore partly offset before and around native execution. A command-card benefit cannot be separated here from fresh verification and receipt handling.

6.3 Activation and secondary observations

Actual captured handoffs match readiness. B and C each preload source on 14 of 24 assignments, covering seven tasks twice, and preserve the original task on the other ten. A passes through all 24. The five abstaining tasks remain in every primary comparison. On the seven source-active tasks, mean uniform equivalents are 0.130432, 0.124227 and 0.108905 dollars for A, B and C; mean workflow times are 39.567, 38.975 and 38.627 seconds. These fixed-stratum summaries are descriptive and receive no subgroup test.

Cost variation also occurs without added source. The largest C-minus-A task difference, -0.069921 dollars per attempt on colorama, occurs in the abstaining stratum. It cannot be interpreted as a direct source-preloading effect. Stochastic agent behavior, shared caches and other uncontrolled execution variation remain plausible contributors; this experiment does not identify their separate effects.

Mean reported whole-attempt input is 85,333 tokens for A, 84,521 for B and 80,328 for C; most input is reported cached. Mean output is 1,743, 1,689 and 1,652 tokens, respectively. These are cumulative native counters, rather than

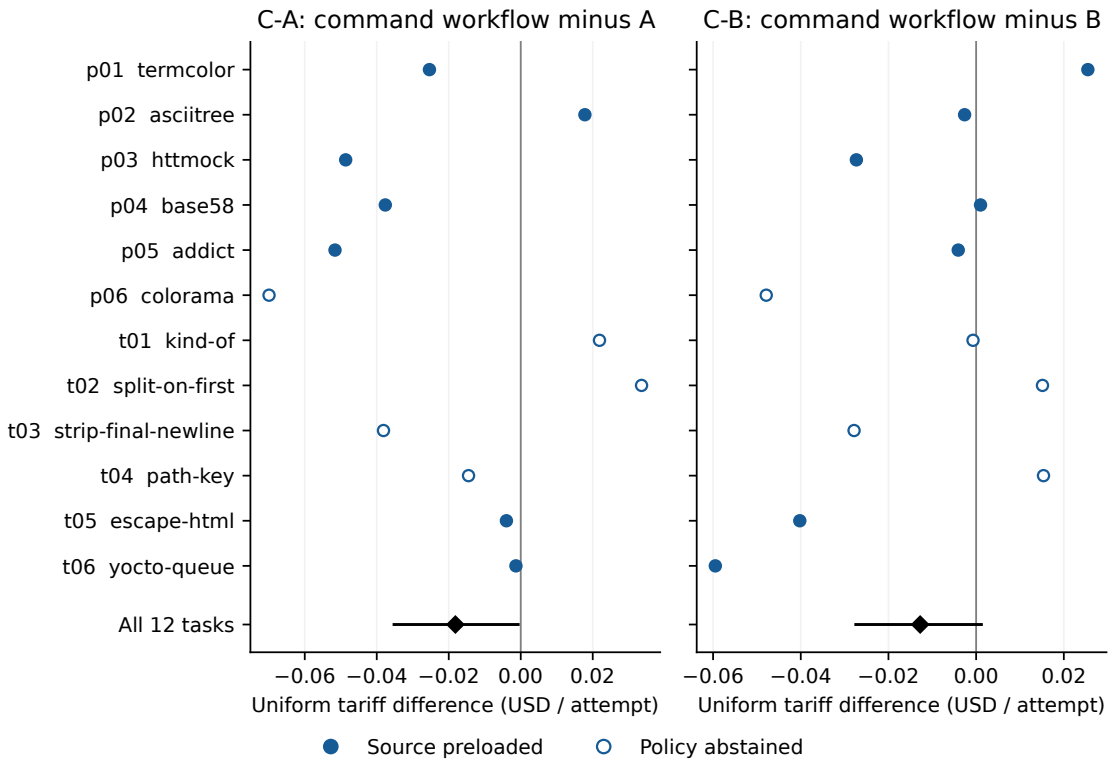


Fig. 2. Task-level cost differences, averaging both repeats before comparison. Filled circles identify the seven source-active tasks; open circles identify the five policy abstentions. Negative values favor C under the fixed uniform tariff. Diamonds and bars show the complete twelve-task mean and pointwise 95% bootstrap interval. The Holm-adjusted exact tests do not reject.

unique source tokens. Under the separate native-price convention, C minus A is -12.49% and C minus B is -8.97%; these secondary calculations remain distinct from the uniform primary metric and from unknown actual invoices. The companion tables retain cached/uncached partitions, cache-write sensitivities and command-text diagnostics. They do not establish avoided filesystem reads or lower provider computation.

A separately implemented automated auditor reproduced all tariff totals, both exact tests, Holm adjustment, bootstrap endpoints and timing scopes from the saved records. It also checked the 72 unique threads, captured task transformations, source/Git protection and raw-to-grade bindings without importing the frozen statistical functions. This automated audit improves internal consistency; it is not external replication.

7 Interpretation and limitations

The experiment measures a context-preparation policy around one installed coding-agent configuration. Source hashes identify supplied bytes; command receipts describe checks on the starting repository; finite grades assess specified behavior; native counters support tariff calculations. None alone establishes better coding in general.

Coverage limits the intervention. Seven of twelve tasks preload at readiness and five abstain. Runtime and declaration files can compete, and source-role classification remains heuristic. The primary intention-to-treat comparison retains

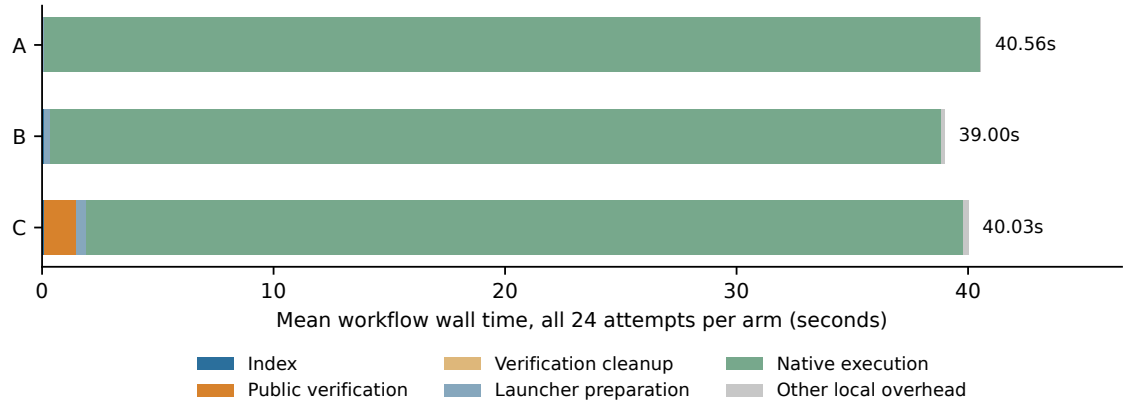


Fig. 3. Mean workflow components over all 24 assigned attempts per arm. C includes fresh public baseline verification and cleanup. Other local overhead is the measured workflow total less the nonoverlapping recorded components. Source copying, grading and packaging are excluded. Native execution includes local process handling and external-model waiting; it is not isolated inference latency.

abstentions and their overhead. Activation strata describe heterogeneity without making the active subset representative. Coordinated runtime/declaration selection or verification deferred until source eligibility would require new evaluation.

The panel is small and purposive, with four JavaScript/TypeScript roots sharing a maintainer. Tasks, references and checks were AI-assisted, and public readiness informed a classification fix before outcomes. Separate agent roles and negative controls do not replace human review or external replication. Base-repository familiarity to the model is unknown. Findings cannot estimate performance on large repositories, historical issues, unfamiliar languages or other models. Task-bootstrap intervals describe panel resampling variation under limited independence assumptions; they do not remove selection bias.

Public, private and declaration checks can miss defects. The base58 oracle cannot prove validation-before-allocation order. Regression-test presence and execution do not establish test quality or full instruction compliance. Matching strict-success counts would not establish population quality equivalence or noninferiority. A provider failure remains a failed workflow, even if a partial patch passes a supplemental check.

The fixed tariff is not an invoice or current-price claim. Actual subscription charges, coordinator effort, engineering, hosting and provider computation are outside the ledger. Agent commands reveal neither internal provider inferences nor all file I/O. Command-text read classifications cannot establish avoided reads. Local CPU and sampled RSS omit remote inference and can miss short peaks. Phase byte caps and the bytes-divided-by-three planning proxy are not model token measurements.

Timing covers fresh projects and run-cache preparation, excluding source/Git copying, grading and packaging. Page caches, provider prefix caches, cloud load and backend model revisions remain uncontrolled. Reversing the schedule balances order but cannot remove persistent carryover. Exact tests assume a sharp null and no cross-arm carryover; remote-provider behavior does not verify these assumptions. Warm-receipt reuse and long-lived project amortization may change the tradeoff but are unmeasured.

Finally, selected source is sent to the external model. Local state placement and heuristic sensitive-content filtering do not make inference local or ensure privacy. Receipts bind checked source, policy, eligible public text and declared

executables, not every dependency or environment setting. The command provider requires authorization and bounds time/output; its product interface does not itself provide operating-system isolation. Stronger grading sandboxes are experimental controls. The source, trust and measurement boundaries in the architecture and methods remain part of interpreting these results.

8 Conclusion

Memglide implements bounded source discovery and optional verified public-command context around an existing coding agent. The installed-system comparison completed all 72 assignments with known usage and passing finite functional, public and integrity checks. Source was supplied on seven of twelve tasks; the policy abstained on five.

The source-and-command workflow had a 13.69% lower observed uniform cost than pass-through and a 10.04% lower cost than source alone. Neither prespecified familywise test established a reduction, and workflow timing did not establish a speed advantage. Fresh verification partly offset the shorter observed native execution. These results support reporting a functioning, auditable implementation and a measured cost estimate with uncertainty. Broader claims require a larger, independently designed task panel and additional models. Coordinated runtime/declaration selection, verification deferred until source eligibility, and amortized receipt reuse are concrete directions for a new evaluation; none is credited with an unmeasured benefit here.

9 Artifact and reproducibility

The companion evidence capsule contains the frozen protocol and allocation, public task fixtures and their upstream provenance, licenses and commands, the exact postclosure-released private grading bundle, captured native prompts and events, token counters, source changes, protection checks, all grading records, analysis source, derived tables and figure sources. Six earlier evidence archives and the historical report inventory are retained with their original hashes. Installed dependencies, native executables, runtime caches and authentication state are excluded. The capsule is an evidence/source artifact, rather than a self-contained execution image.

The supplement documents the environment, source change, control runs, measurement boundaries and reproduction commands. Archive verification and statistical reproduction require no model calls. Repeating native collection requires the declared tools, a separately authenticated account and a new study allocation; the completed allocation rejects retries and overwrites. The project repository is https://github.com/iamsreecharan/Memglide. This prepared manuscript and companion capsule have not been deposited under a public archive DOI, submitted to a venue or independently replicated.

10 AI assistance

OpenAI Codex assistants supported implementation, task and control generation, experiment orchestration, analysis code, literature-source checks and manuscript drafting. Separate agents reviewed task contracts and experimental records. These automated reviews do not constitute independent human replication. The coding model used as an experimental subject is specified in the methods. No AI tool is listed as an author. Verification of the final manuscript and responsibility for any submission rest with the named author, Sree Charan Reddy Kailasam.

References

[1] Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2023. LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 13358–13376. doi:10.18653/v1/2023.emnlp-

main.825

[2] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-world Github Issues?. In *The Twelfth International Conference on Learning Representations*. https://openreview.net/forum?id=VTF8yNQM66

[3] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics* 12 (2024), 157–173. doi:10.1162/tacl_a_00638

[4] Tianyang Liu, Canwen Xu, and Julian McAuley. 2024. RepoBench: Benchmarking Repository-Level Code Auto-Completion Systems. In *International Conference on Learning Representations*. https://arxiv.org/abs/2306.03091

[5] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2024. Agentless: Demystifying LLM-based Software Engineering Agents. *arXiv preprint arXiv:2407.01489* (2024). https://arxiv.org/abs/2407.01489

[6] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik R Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*. https://arxiv.org/abs/2405.15793

[7] Fengji Zhang, Bei Chen, Yue Zhang, Jacky Keung, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Weizhu Chen. 2023. RepoCoder: Repository-Level Code Completion Through Iterative Retrieval and Generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 2471–2484. doi:10.18653/v1/2023.emnlp-main.151